

Cron at the center of your pipelines

A "stuck" pipeline is worse than a failing one

The context

Do you know the silent villain? The task that never finishes: an API call stuck on the client library's default timeout, an `ssh` waiting for a password, a `pandas.read\_csv` against an S3 that takes 40 minutes. With Cron or loose scripts, that job stays there forever — consuming RAM, holding a slot, and leaving everyone wondering: did it finish? Did it crash?

Why engineers shift to wpipe

- Max time per step defined in code (versionable).
- No production surprises: you know how long each stage can take.
- Timeout failures are recorded in the SQL tracker with their context.

Code-first

Pipelines are Python: versionable, testable, reviewable in Git.

Checkpoints

State persisted to SQLite WAL at every step; resume, don't restart.

Independently lean

<50MB RAM. No broker, no daemon, no metadata server.

What it looks like

```
from wpipe import Pipeline, step

@step(name="job", retry_count=3, retry_delay=10)
def job(data):
    return {"done": run(data["command"])}

pipe = Pipeline(pipeline_name="cron_pro", tracking_db="cron.db")
pipe.set_steps([job])

# Keep the schedule; gain checkpoints + audit.
```

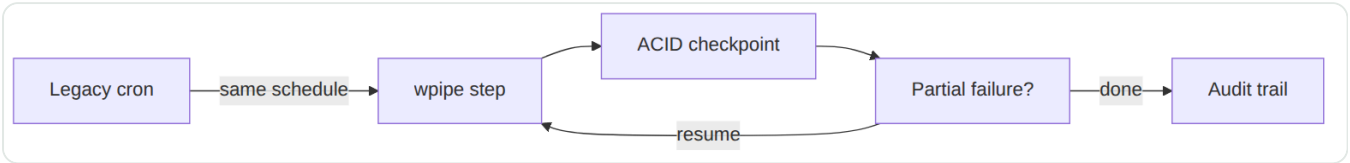
At a glance

Aspect	Legacy / Cron	wpipe
Definition	Config/DSL or canvas	Python code
State	Scattered / none	SQLite WAL checkpoints
Observability	Partial	Tracker SQL + dashboard

Aspect	Legacy / Cron	wpipe
Footprint	Heavy	<50MB RAM

Under the hood

The flow behind the post, as wpipe renders it from code.



+117k downloads
A community of efficiency-first developers.

<50MB RAM
Fits containers, VMs, and Raspberry Pi-class hardware.

SQLite WAL
Industrial durability without a database to administer.

Next step

wpipe is a Python library — try it in your next pipeline:

```
pip install wpipe
```

Steps stay functions you already understand. Docs, diagrams, and dashboards follow. Explore Cron workflows at wpipe.readthedocs.io.

wpipe · Python-native orchestration with SQLite WAL checkpoints, retries, timeouts, parallelism, and self-documenting flows. +117k downloads.